Project Deadline: 23.10.2025 23:59

CEN322 – Network Programming

Project 01

Network Diagnostic and Tool Package Project

1. Project Objective

The purpose of this project is to provide students with hands-on experience in basic socket programming, network diagnostics, time synchronization, and error management. In the project, small applications (such as machine information extraction, echo testing, SNTP client, socket settings, and error management), which are developed separately, will be integrated to create a comprehensive network diagnostic tool. Through this project, students will experience the fundamental network operations and troubleshooting methods that they may encounter in real-world scenarios.

---

2. Required Tools and Prerequisites

Programming Language: Python 3

Potentially Required Libraries:

- socket
- time (or a third-party library related to SNTP such as ntplib – optional)
- argparse (optional, for command-line parameters)
- Other standard Python modules (e.g., json, sys)

Development Environment:

Preferably, a Linux distribution or an environment with Python 3 installed on Windows/Mac.

Note: You can work using a virtual environment (virtualenv).

---

3. Project Structure and Modules

The project will consist of the following four main modules:

A. Machine Information Module

Objective:
Obtain the local machine's hostname, IP address, network interfaces, and related information.

Tasks:

- Retrieve the hostname.
- List the network interfaces.
- Obtain the device's IP address(es).

Output:
A function that prints or logs the machine information.

B. Echo Test Module

Objective:
Perform basic connection tests by setting up a simple echo client/server structure.

Tasks:

- On the server side, create a socket and listen on a specified port.
- On the client side, connect to the server and receive the same data that was sent.
- Compare the sent and received data.

Output:
Display the connection test result on the screen (for example, "Connection successful, data matches").

C. SNTP Time Synchronization Module

Objective:
Retrieve the current time from an SNTP (Simple Network Time Protocol) server on the Internet.

Tasks:

- Send a request to an SNTP server (for example, pool.ntp.org).
- Parse the time from the response (either by processing raw data or using a library like ntplib).
- Compare the obtained time with the local system time or print the time information.

Output:
Print the current time information received from the server.

D. Simple Chat Module

Objective:
Provide a basic chat environment for the user.

Tasks:

- Allow the user to choose whether to run in server or client mode.
- Establish the connection.
- Transfer strings over the established connection and display them on the screen.

Output:
Display chat messages on the screen and save output to a file.

E. Error Management and Settings Module

Objective:
Experience socket timeout, buffer size, blocking/non-blocking mode settings, and error management.

Tasks:

- Modify socket timeout, send/receive buffer sizes.
- Switch between blocking and non-blocking modes.
- Catch potential error situations (e.g., connection errors, timeouts) and display appropriate error messages.

Output:
Log or print the applied settings and any encountered errors.

---

4. Project Integration

Students will combine the modules defined above into a single integrated application. The following steps will be followed in the project:

Main Menu Design:

- An interactive menu will be presented to the user, allowing them to choose which module to run.
- **Menu options:** Machine Information, Echo Test, SNTP Time Check, Socket Settings, Chat, and Error Management.

Modular Approach:

- Each module will be designed as a separate Python function or class.
- The main program will call the relevant module based on the user's choice.

Data Integration:

- The results obtained from the Echo Test module may be linked with the Error Management module.
- The time information from the SNTP module can be used in other modules for logging or time comparison.

---

5. Deliverables

- **Code Files:** All Python files that constitute the project (including libraries).
- **Documentation:** Comments within the code explaining each function and/or a separate README file.
- **Execution Guide:** Instructions on how to run the project, which libraries need to be installed, and usage examples.
- **Test Outputs:** Screen outputs or log files demonstrating that each module works correctly.
- **All files/documents should be compressed into a single file (zip/rar etc.). Do NOT upload multiple files.** Send your file to sbilgili@cu.edu.tr Do not forget to include your student ID, first and last name.

---

6. Evaluation Criteria

- **Code Quality:** Clear, readable, and modular code.
- **Functionality:** All modules should produce the expected outputs.
- **Error Management:** Proper handling of potential errors and informative error messages.
- **Documentation:** Detailed comments in the code and a comprehensive execution guide.
- **Creativity:** Additional features integrated into the project (e.g., a user-friendly interface, extra reporting functionality).

---

7. Additional Suggestions

- **During Development:**
  It is recommended that students develop and test each module separately before integrating them. This way, each module's functionality can be verified individually.
- **Version Control:**
  You may use tools like Git for project development and source code management.
- **Feedback:**
  Regular reviews and feedback during the mid-term phase, followed by a comprehensive final evaluation.

Last important note:

- **Plagiarism / AI will not be tolerated.**
- **You are free to use AI tools for ONLY GUI.**